

Computación Paralela y Distribuida

2026-I

José Fiestas PhD

3 de junio de 2026

Universidad de Ingeniería y Tecnología
jfiestas@utec.edu.pe



Unidad 4: Comunicación y coordinación

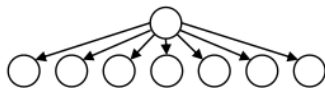
Objetivos:

1. Pasos de Mensaje: MPI, Mensajes Punto a Punto, MPI, Comunicación Colectiva, Blocking vs non-blocking
2. Memoria Compartida: OMP, Constructores y cláusulas, CUDA, optimizacion con GPUs
3. Programacion Hibrida

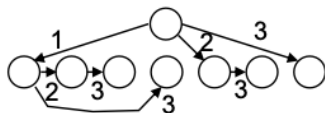
Message Passing Interface

Comunicación colectiva

E.g. broadcast:



Sequential algorithm
 $O(\# \text{ processes})$



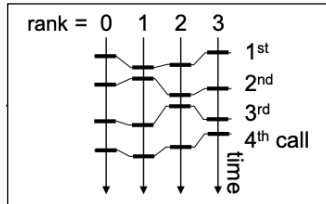
Tree based algorithm
 $O(\log_2(\# \text{ processes}))$

La librería MPI se encarga de optimizar la comunicación

Comunicación colectiva

Propiedades:

- Facilita operaciones colectivas de comunicación sin interferir con comunicación punto a punto
- Todos los procesos en un comunicador participan de comunicación colectiva
- En un comunicador cada llamada colectiva debe incluir a todos los procesos
- El buffer se re-utiliza solo cuando el proceso termina



MPI_Barrier:

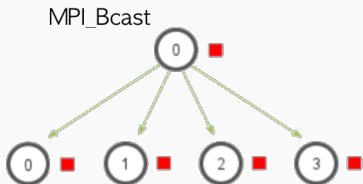
Operación de sincronización. No utiliza data. Funciona bloqueando el proceso de llamada hasta que todos los miembros del grupo han llamado a la operación

MPI_Barrier(comm)

Normalmente, MPI_Barrier no se necesita, ya que la sincronización es automática (procesos no pueden continuar sin su data). Util para debugging, pero debe ser removida luego.

MPI_Bcast:

Se envía mensaje del rank
'root' a todos los
procesos en el grupo

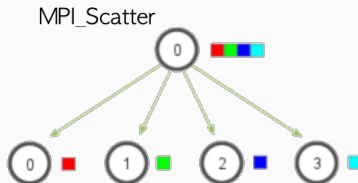


MPI_Bcast(&buf, count,datatype,root,comm)

root es el rank del proceso que envía el mensaje.

MPI_Scatter:

Distribuye mensajes de una sola fuente a cada proceso según el orden del rank



MPI_Scatter(&buf,sendcnt,sendtype,&recvbuf,recvnt,recvtype,root,comm)

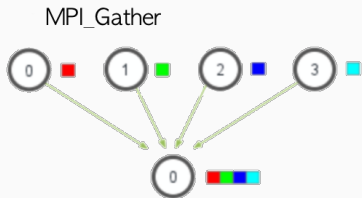
- **&buf**: dirección del buffer que reside en el nodo raíz
- **sendcnt**: número de elementos a ser enviados por proceso (elementos del array / número de nodos)
- **sendtype**: tipo de elementos enviados
- **&recvbuf**: buffer que recibe **recvnt** elementos del tipo **recvtype**
- **root**: nodo raíz
- **comm**: comunicador en el que residen los procesos

MPI_Gather:

Recopila información de cada proceso en un solo destino.

Reverso de MPI_Scatter.

Los elementos están ordenados de acuerdo al rango del proceso de donde son recibidos



MPI_Gather(&sendbuf,sendcnt,sendtype,&recvbuf,recvnt,recvtype,root,comm)

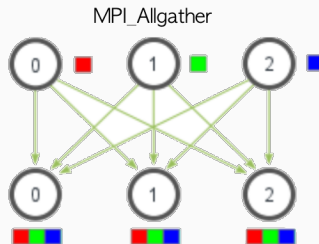
- **&sendbuf**: dirección del buffer que reside en el nodo raíz
- **sendcnt**: número de elementos a ser enviados por proceso
- **sendtype**: tipo de elementos enviados
- **&recvbuf**: buffer que recibe **recvnt** elementos del tipo **recvtype**
- **root**: nodo raíz
- **comm**: comunicador en el que residen los procesos

MPI_Allgather:

Recopila información de cada proceso en todos los demás procesos. Actúa como un MPI_Gather seguido de un MPI_Bcast

Los elementos son recopilados en el orden de rango de donde provienen

No contiene un proceso principal (root)

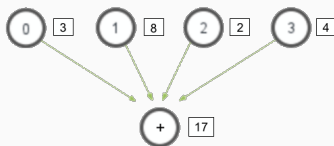


```
MPI_Allgather(&sendbuf,sendcnt,sendtype,&recvbuf,recvnt,  
recvtype,comm)
```

MPI_Reduce:

Recopila los datos del grupo y los pone en un task

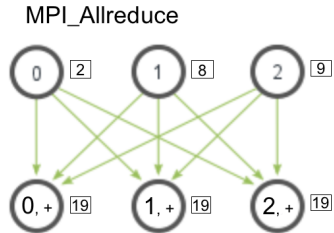
MPI_Reduce



MPI_Reduce(&sendbuf,&recvbuf,count,datatype,op,root,comm)

MPI_Allreduce:

Recopila los datos del grupo y los pone en todos los task del grupo



```
MPI_Allreduce(&sendbuf,&recvbuf,count,datatype,op,comm)
```

Operaciones de reducción

Operadores pre-definidos

MPI Name	Function
MPI_MAX	Maximum
MPI_MIN	Minimum
MPI_SUM	Sum
MPI_PROD	Product
MPI LAND	Logical AND
MPI_BAND	Bitwise AND
MPI_LOR	Logical OR
MPI_BOR	Bitwise OR
MPI_LXOR	Logical exclusive OR
MPI_BXOR	Bitwise exclusive OR
MPI_MAXLOC	Maximum & location
MPI_MINLOC	Minimum & location

Ejemplo 01: Bcast y reducción

```
1  int main(int argc, char*argv[])
2  {
3      MPI_Init(&argc,&argv);
4      MPI_Comm_rank(MPI_COMM_WORLD, &rank);
5      MPI_Comm_size(MPI_COMM_WORLD, &size);
6      ...
7      MPI_Bcast(&data,1,MPI_INT,0,MPI_COMM_WORLD);
8      ...
9      MPI_Reduce(&data,&res,DATA_SIZE,MPI_INT,MPI_SUM,0,MPI_COMM_WORLD);
10     ...
11     if(me==0){
12         // imprimir resultados en maestro
13     }
14     MPI_Finalize();
15 }
```

Ejemplo 02: Scatter/Gather

```
1  int main(int argc, char*argv[])
2  {
3      MPI_Init(&argc,&argv);
4      MPI_Comm_rank(MPI_COMM_WORLD, &rank);
5      MPI_Comm_size(MPI_COMM_WORLD, &size);
6      ...
7      if(rank==0){
8          //inicializar array globaldata[np] en proceso 0
9      }
10     MPI_Scatter(&globaldata,1,MPI_INT,&localdata,1,MPI_INT,0,MPI_COMM_WORLD);
11     ...
12     // multiplicar localdata por 2
13     MPI_gather(&localdata,1,MPI_INT,&globaldata,1,MPI_INT,0,MPI_COMM_WORLD);
14     ...
15     if(me==0){
16         // imprimir globaldata en maestro
17     }
18     ...
19     MPI_Finalize();
20 }
```

Ejemplo 4: Cálculo de Pi por integración

Calcule el valor de PI en paralelo a través de la fórmula de aproximación

$$\pi = \int_0^1 \frac{4}{1+x^2} dx = \frac{1}{n} \sum_{i=1}^n \frac{4}{1 + \left(\frac{i-0.5}{n}\right)^2}$$

- Utilice MPI_Bcast para distribuir el número de intervalos n a integrar a todos los procesos. Variar n de 10^2 a 10^6 en potencias de 10. Asigne correctamente los límites de integración para cada proceso
- Utilice MPI_Reduce para recolectar el cálculo de cada proceso y sumarlo en el resultado de Pi. Calcule el error con respecto a 3.141592653589793238462643
- Utilice MPI_Wtime() para medir tiempos en combinaciones de n y p , donde $10^2 < n < 10^6$, $1 < p < 8$ y graficarlos en función a p
- Analice los resultados obtenidos

Ejemplo 5: Cálculo del promedio

Calcule el promedio de números en un array $n[1000]$. Para ello:

- Genere un array con números random de 0 a 100
- Asigne a cada proceso una cantidad equivalente de números (MPI_Scatter)
- Cada proceso calcula el promedio de su muestra
- Se agrupan los promedios en un array de sumas (MPI_Gather) en el nodo principal, y éste calcula el promedio global
- Haga variar el número de procesos p de 2 a 40
- Mida y grafique tiempos de ejecución en función a p
- Analice los resultados obtenidos

-  <https://www.mpich.org/documentation/guides/>
-  Peter S. Pacheco. *An Introduction to Parallel Programming*. 1st. Morgan Kaufmann, 2011. isbn: 978-0-12-374260- 5. Ch 3
-  Michael J. Quinn. *Parallel Programming in C with MPI and OpenMP*. 1st. McGraw-Hill Education Group, 2003. isbn: 0071232656. Ch 4
-  William Gropp, Ewing Lusk and Anthony Skjellum: *Using MPI: Portable Parallel Programming With the Message-Passing Interface*. MIT Press, 3rd edition, Nov. 2014 (336 pages, ISBN 9780262527392)
-  William Gropp, Torsten Hoefler, Rajeev Thakur and Ewing Lusk: *Using Advanced MPI: Modern Features of the Message-Passing Interface*. MIT Press, Nov. 2014 (392 pages, ISBN 9780262527637)



Neil MacDonald, Elspeth Minty, Joel Malard, Tim Harding, Simon Brown, Mario Antonioletti: *Parallel Programming with MPI. Historical MPI course notes from EPCC.*

http://www.archer.ac.uk/training/course-material/2014/10/MPI_UCL/Notes/MPP-notes.pdf

